



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ОТЧЕТ

Домашнему заданию №3

НА ТЕМУ:

*«Разработка приложения на языке C# с базой
данных и пользовательским интерфейсом»*

Студент

ИУ5-21Б

(группа)

(подпись, дата)

С.Г. Ерицян

(И.О. Фамилия)

Руководитель

(подпись, дата)

Ю.Е. Гапанюк

(И.О. Фамилия)

2026 г.

Цель работы: проектирование и разработка настольного приложения на языке C# с использованием графического интерфейса Windows Forms и СУБД SQLite для хранения данных о музыкальных альбомах и треках. В работе используются технологии ORM Entity Framework Core для взаимодействия с базой данных и LINQ для выполнения аналитических запросов.

Назначение программы: программа предназначена для систематизации музыкальной коллекции. Она позволяет:

- Хранить данные об альбомах и входящих в них песнях.
- Фильтровать список треков по названию.
- Формировать отчеты: подсчет количества треков в альбомах и расчет средней длительности треков.

Структура программы: программа построена по принципу разделения ответственности:

- **Модели (Album, Song):** Классы, описывающие структуру БД.
- **Контекст БД (AppDbContext):** Класс, наследующий DbContext, конфигурирует базу music_collection.db и инициализирует её начальными данными (Seed Data).
- **Интерфейс (MainForm):** Окно приложения, содержащее DataGridView для отображения данных и кнопки для вызова аналитических LINQ-запросов.

Описание работы программы: при запуске Database.EnsureCreated() создает SQLite-файл и наполняет его данными.

- **Поиск:** выполняется через `.Where(s => s.Name.Contains(searchText))`.
- **Аналитический запрос 1 (Количество треков):** группировка через `GroupBy(s => s.Album!.Name)` с применением агрегатной функции `.Count()`.
- **Аналитический запрос 2 (Средняя длительность):** группировка по альбому с расчетом `.Average(s => s.DurationSec)` и сортировкой `.OrderByDescending()`.

Результат выполнения программы

Раздел 1. Все песни (Основная + Справочная)				
	ID	Название	Длительность_С	Альбом
▶	1	Time	401	The Dark Side o...
	2	Money	382	The Dark Side o...
	3	Us and Them	462	The Dark Side o...
	4	Come Together	259	Abbey Road
	5	Something	182	Abbey Road
	6	Hells Bells	312	Back in Black
	7	Back in Black	255	Back in Black
	8	You Shook Me ...	210	Back in Black

Рисунок 4. Главное окно с таблицей всех песен

Поиск песни: Something Найти				
Раздел 1. Все песни (Основная + Справочная)				
	ID	Название	Длительность_С	Альбом
▶	5	Something	182	Abbey Road

Рисунок 3. Результат фильтрации по поисковому запросу

Аналитические отчеты (Разделы 2 и 3)		
Отчет 1: Треков в альбоме		Отчет 2: Средняя длительность
	Альбом	Количество_Тре
▶	Abbey Road	2
	Back in Black	3
	The Dark Side o...	3

Рисунок 2. Результат отчета "Количество треков в альбоме"

Аналитические отчеты (Разделы 2 и 3)		
Отчет 1: Треков в альбоме		Отчет 2: Средняя длительность
	Альбом	Средняя_Длительность_Сек
▶	The Dark Side o...	415
	Back in Black	259
	Abbey Road	220,5

Рисунок 1. Результат отчета "Средняя длительность треков"

Выводы: в ходе работы освоены принципы объектно-реляционного отображения (ORM) и возможности языка LINQ. Применение Include позволило эффективно загружать связанные данные, а группировка данных по навигационным свойствам доказала свою лаконичность для построения отчетов. Программная архитектура на базе .NET 10 обеспечивает высокую стабильность и простоту расширения функционала.